PayShield — Comprehensive Project Documentation

This document serves as the ultimate source of truth for the **PayShield** project, built for Hack-Nocturne 2.O. It details the problem, architecture, ML engine, exact technical implementation, and demo flow.

📖 Table of Contents

1. Executive Summary
2. The Problem Deep Dive
3. The PayShield Solution
4. Architecture & Services
5. Data Flow Lifecycle
6. Machine Learning Engine (XGBoost)
7. API & Security Implementation
8. The Requestly Integration (Live Attack)
9. Deployment & Running

---

1. Executive Summary

PayShield is an AI-powered Payment Fraud Intelligence System designed specifically to detect and block **Client-Side API Tampering** in real-time. By combining lightning-fast Edge Middleware, an XGBoost Machine Learning fraud engine, and a live WebSocket-powered replay dashboard, PayShield ensures that requests leaving the user's browser are the exact same requests the server processes.

2. The Problem Deep Dive

What is Client-Side API Tampering? Modern web applications execute business logic (like deciding the price of an item) on the client side (browser/app) and send the final result to the server via API calls.

Attackers exploit this "client trust" by intercepting the network request right after the user clicks "Pay" but before it leaves the device. Using tools like browser DevTools, Burp Suite, or extensions like **Requestly**, attackers can:

- **Modify the payload:** Change `amount: 15000` to `amount: 1`
- **Modify the destination:** Swap the `merchant_id` to their own account.
- **Inject headers:** Bypass authentication or spoof IP origins.

Why existing systems fail:

- **WAFs (Web Application Firewalls)** block SQL injection and cross-site scripting (XSS), but they don't know the business logic of a payment.
- **Traditional Fraud Engines** analyze the data *after* it has arrived. If a transaction says "amount: 1", the engine assumes the user actually bought a ₹1 item, unaware it was tampered with.

3. The PayShield Solution

PayShield assumes zero trust between the client UI and the API. It defends across three layers:

1. **The Edge (Detection):** Custom Express.js middleware inspects `req.headers` and `req.body` for tampering signatures (e.g., proxy injection headers) in `<2ms`.
2. **The Brain (Scoring):** Even if tampering is found, the transaction is routed to a Python ML backend that evaluates 6 historical features to provide an independent risk score for auditing.

3. **The Eyes (Visibility):** The outcome is instantly broadcasted via Redis WebSockets to a React/Next.js dashboard, transforming complex logs into a visual, step-by-step pipeline called the **Replay Engine**.

---

4. Architecture & Services

The system is built on a 4-container microservice architecture.

Service	Technology	Port	Purpose
API Gateway	Node.js + Express + Redis	`3000`	Entry point. Handles routing, JWT auth, rate limiting (30/min), idempotency caching, and HTTP proxying to the Fraud Engine.
Fraud Engine	Python + FastAPI + Scikit-Learn	`8000`	Machine Learning inference server. Houses the pre-trained XGBoost model.
Dashboard	Next.js + React + Tailwind	`4000`	Real-time frontend. Connects to the Gateway via `ws://` to stream live events.
Redis	Redis (Alpine)	`6379`	In-memory message broker (Pub/Sub) for WebSockets and state cache for the Gateway.

---

5. Data Flow Lifecycle

How a single payment traverses the PayShield architecture:

1. **Browser** sends `POST /api/v1/payment/initiate` to the Gateway.
2. **Gateway Middleware (req, res, next):**
   - Applies `uuidv4` Request ID.
   - Parses JWT token (Authentication).
   - Checks Redis for Rate Limit (`<30 requests/min`).
   - Checks Redis for Idempotency Key (prevents double charging).
   - **Tamper Detection:** Scans headers. If `X-Requestly-Modified: true` is found, flags `req.tampered = true` .
3. **Gateway** sends HTTP POST to Fraud Engine over internal Docker network.
4. **Fraud Engine** evaluates features through the XGBoost model and returns `risk_score` (0.0 to 1.0) and `label` (LOW/MEDIUM/CRITICAL).
5. **Gateway Decision Logic:**
   - If `req.tampered == true` 👉 Immediate **BLOCK (402 FRAUD_FLAGGED)**.
   - If `risk_score >= 0.8` 👉 **BLOCK (402 High Risk)**.
   - Else 👉 **ALLOW (200 OK)**.
6. **Pub/Sub:** Gateway publishes the final JSON event to the Redis `tx:live` channel.
7. **WebSocket Broadcaster:** Reads from Redis and pushes to all connected Dashboard clients.
8. **Dashboard UI:** Renders the transaction in the live feed and updates the Replay Engine UI.

---

6. Machine Learning Engine (XGBoost)

The Brain of the operation runs on **XGBoost** deployed via FastAPI.

Why XGBoost? Unlike deep learning, gradient-boosted decision trees (XGBoost) handle tabular financial data exceptionally well with near-instant inference times (<50ms).

Features Evaluated in Real-Time:

1. Transaction Amount variance
2. Card BIN risk profile
3. Device fingerprint history
4. Geographical distance (IP to Country mapping)
5. Time of day anomalies
6. Velocity (transactions per IP in last hour)

Note: For the scope of this hackathon demo, the model simulates these specific statistical distributions based on a seeded dataset.

---

7. API & Security Implementation

Key Security Engineering Practices Implemented:

- **Idempotency Keys (`X-Idempotency-Key`):** Stored in Redis for 24 hours. If a user double-clicks the "Pay" button, the second identical request is blocked to prevent double-charging.
- **JWT Rotation:** Tokens expire and are rotated every 15 minutes.
- **Rate Limiting (Sliding Window):** Allows exactly 30 requests per IP per 60,000ms window to prevent brute forcing.
- **Helmet.js:** Secure HTTP headers (XSS Filter, HSTS, No-Sniff).
- **Masking:** Credit card BINs are truncated and PII is scrubbed before being logged or forwarded to the WebSocket.

---

8. The Requestly Integration (Live Attack)

PayShield's core novelty is proving its efficacy by attacking itself live. We use **Requestly**, a browser extension designed for developers to intercept network traffic.

The Attack Vector:

1. A legitimate user clicks "Test Tamper" on the dashboard.
2. The browser attempts to send a clean payload: `{amount: 2500, currency: "INR"}`
3. The Requestly extension intercepts this HTTP request mid-flight locally on the machine.
4. Requestly injects a custom header: `X-Requestly-Modified: true` to simulate proxy tampering.
5. The request leaves the machine and hits the API Gateway.
6. The Gateway's `tamperDetection()` middleware catches the specific header signature, instantly proving the backend can detect invisible client-side manipulation.

This setup proves that the defense mechanisms work against actual browser-level intercepts, not just mocked database flags.

---

9. Deployment & Running

The system requires only Docker to run, eliminating "it works on my machine" issues for judges.

```
# Setup network and containers
docker-compose up -d --build
```

```
# Generate 25 live demo transactions (including 1 tampered payload)
cd gateway
npm install
node src/scripts/seedDemo.js
```

Everything is mapped to `localhost` ports defined in Section 4. The system is designed to cold-start and be fully operational within 15 seconds.